CIFAR-10 + ResNet-18 Test-Time Training (TTT) Plan

TER M1 Vision et Machine Intelligente

March 18, 2026

1 Foundations (from basics)

This section defines the core terms used later in the document. Each part is standalone and written from first principles.

Feed-forward model

A feed-forward model processes an input once, from layer to layer, with no loops. It does not keep a memory across time steps; it simply maps the input to an output.

Hidden state

A hidden state is a small memory vector used by recurrent models. It carries information from one time step to the next in a sequence.

RNN (Recurrent Neural Network)

An RNN reads data step-by-step (for example, a sentence word by word), updating its hidden state at each step. It can struggle to keep information from very early steps because gradients can vanish over long sequences.

CNN (Convolutional Neural Network)

A CNN is designed for images. It learns local patterns (edges and textures) using convolutional filters and then combines them into higher-level shapes and objects. It processes the image in one forward pass.

Convolution

A convolution is a small filter (kernel) that slides across the image and computes a weighted sum of local pixels. The same filter is reused everywhere, which makes learning efficient and captures repeated patterns.

Multi-branch block

A multi-branch block sends the same input through several parallel paths (for example, different filter sizes) and then merges the results. This helps the model capture patterns at multiple scales.

FLOP

A FLOP is a single floating-point operation (such as a multiply or add). FLOPs are a rough measure of computational cost; fewer FLOPs usually means faster inference.

Backbone and head

The backbone is the main feature extractor. In this project it is ResNet-18. A head is a small module attached to the backbone for a specific task, such as classification or rotation prediction.

Classification

Classification assigns an input to one of a fixed set of categories. The model outputs scores (logits) for each class; after a softmax, these become probabilities, and the highest probability is the prediction.

Loss and cross-entropy

A loss measures how wrong a prediction is. For classification, cross-entropy compares the predicted probabilities to the true label and penalizes confident mistakes.

Backpropagation

Backpropagation computes gradients of the loss with respect to the model parameters. An optimizer then updates the weights to reduce the loss.

Self-supervised task (pretext task)

A self-supervised task creates its own labels from the data (for example, predicting image rotation). It allows the model to learn useful features without human annotation.

Fine-tuning

Fine-tuning means taking a pretrained backbone and continuing training on a labeled task. This adapts the learned features to the new dataset.

Transformer and self-attention

A vision transformer splits an image into patches (tokens) and uses self-attention so each patch can directly interact with all other patches. This captures global relationships in the image.

Fusion and decoder

Fusion combines information from multiple branches or modalities (for example, RGB and depth). A decoder turns a representation back into a structured output (for example, a segmentation map). Our task is classification, so we do not need fusion or a decoder.

Distribution shift

Distribution shift means the test data differs from the training data (for example, rotated images). This often reduces accuracy.

Test-time training (TTT)

TTT adapts the model on unlabeled test data using a self-supervised loss. It updates part of the backbone to better match the test distribution without using class labels.

2 Deeper academic notes (still simple)

This section adds more formal context while keeping the explanations readable.

Supervised classification in notation

Let an image be x and its label be $y \in \{1, \ldots, K\}$. A classifier produces logits $z = f_\theta(x)$, then probabilities $p = \mathrm{softmax}(z)$. The prediction is $\arg\max_k p_k$. Training minimizes the average loss over a labeled dataset.

Why cross-entropy is the standard loss

Cross-entropy matches the probabilistic interpretation of classification. If p is the predicted distribution and y is the true class, the loss $\mathcal{L}_{\mathrm{cls}} = -\log p_y$ penalizes confident wrong predictions more heavily than uncertain ones. This aligns with maximum-likelihood estimation.

Self-supervised rotation as a pretext task

Rotation prediction creates labels from the data itself (0, 90, 180, 270 degrees). The model must understand object geometry to solve the task, so the backbone learns features that generalize better than random initialization.

TTT objective in words

During test time, we do not change the classifier head. We only adapt the backbone using the rotation loss on unlabeled test images. This is a small, controlled update that nudges features toward the test distribution without seeing labels.

Why this is not label leakage

No ground-truth class labels are used at test time. The only supervision comes from the rotation labels that we create ourselves. This is allowed because those labels do not reveal the true class.

Distribution shift: types you can mention

Covariate shift means $P(X)$ changes while $P(Y|X)$ stays stable. Label shift means $P(Y)$ changes. Domain shift means both may change (different style or capture conditions). In our case, rotation changes image geometry but keeps the semantic label, so it is mainly a covariate shift.

Evaluation logic

We report clean accuracy (no rotation) to confirm the base model is healthy, and rotated accuracy to measure robustness. The difference between TTT and no-TTT on rotated data isolates the benefit of test-time adaptation.

What parameters to update during TTT

Updating only the backbone (or only its last blocks) stabilizes adaptation and reduces overfitting to small test batches. Keeping the classifier head fixed makes results easier to interpret.

3 CNNs vs Transformers (with examples)

CNNs use convolutional filters with local receptive fields and weight sharing, which makes them efficient and effective on smaller datasets. Vision Transformers (ViTs) split an image into patches and use self-attention so every patch can interact with every other patch, which gives them strong global modeling but often requires more data or regularization.

Conceptually, CNNs build features from edges to textures to object parts, while ViTs operate on patch tokens and learn relationships across all tokens.

Example (PyTorch code, CNN vs ViT forward pass):

```
import torch
from torchvision import models

x = torch.randn(4, 3, 224, 224) # batch of images

# CNN backbone
cnn = models.resnet18(weights=None)
cnn_logits = cnn(x) # [4, 1000]

# Transformer backbone
vit = models.vit_b_16(weights=None)
vit_logits = vit(x) # [4, 1000]
```

The TTT idea is the same for both backbones: add a self-supervised head (rotation prediction) and update the backbone on unlabeled test images. Only the backbone architecture changes.

4 Other CNN model families (short guide)

Each paragraph gives the core idea and the main trade-offs in plain language.

LeNet-5

A very small, early CNN with only a few convolutional layers. It is fast and great for teaching, but it is far too small for modern accuracy.

AlexNet

An early deep CNN with large kernels and ReLU activations. Historically important, but heavy and less accurate than modern models.

VGG (11/13/16/19)

A simple, uniform design built from many 3×3 convolutions. Easy to explain, but computationally heavy.

ResNet (18/34/50/101/152)

Uses residual (skip) connections so very deep networks can train reliably. A strong, widely used baseline, but large variants are heavy.

WideResNet

Makes residual blocks wider instead of much deeper, which is effective on CIFAR. Accurate and stable, but larger in parameter count.

DenseNet

Connects each layer to all previous layers to encourage feature reuse. Accurate, but memory intensive.

Inception (GoogLeNet)

Uses multi-branch blocks that capture features at multiple scales. Efficient, but more complex to implement.

ResNeXt

Adds grouped convolutions (cardinality) to residual blocks. Scales well and is accurate, but more complex than ResNet.

SqueezeNet

A very compact CNN with "fire" modules. Tiny and fast, but sacrifices accuracy.

MobileNet (v1/v2/v3)

Uses depthwise separable convolutions for mobile efficiency. Lightweight and fast, with some accuracy trade-off.

ShuffleNet

Uses group convolutions and channel shuffling for speed on mobile hardware. Efficient, but less accurate than larger CNNs.

EfficientNet

Scales depth, width, and resolution in a balanced way. Strong accuracy per FLOP, but harder to tune and reproduce.

RegNet

A systematic CNN design space with predictable scaling rules. Accurate and fast, but less intuitive for teaching.

SENet / SE-ResNet

Adds channel attention (squeeze-and-excitation) to improve accuracy with small overhead, but adds extra modules.

ConvNeXt

A modern CNN with transformer-inspired design choices. Very strong, but heavier than ResNet-18.

Practical note for CIFAR-10. ResNet-18 and WideResNet are the most common balanced choices because they are stable, accurate, and easy to explain.

5 Practical setup (CIFAR-10 in this repo)

Download CIFAR-10 into `data/raw/`. The extracted folder must be `data/raw/cifar-10-batches-py` for torchvision to find it.

To keep the path identical for all teammates, run: `python scripts/download_cifar10.py`. It downloads and verifies the dataset in the correct folder.

When using `torchvision.datasets.CIFAR10`, set `root="data/raw"` and `download=False`. Torchvision will look for `data/raw/cifar-10-batches-py`.

Code example (CIFAR-10 load + check):

```
from torchvision import datasets, transforms

transform = transforms.ToTensor()
train_set = datasets.CIFAR10(
    root="data/raw",
```

```
    train=True,
    download=False,
    transform=transform,
)

print(len(train_set))
print(train_set[0][0].shape, train_set[0][1])
```

Result (expected):

```
50000
torch.Size([3, 32, 32]) <label_id>
```

6 Goal and alignment with the TER guide

The TER guide recommends a pipeline that is defensible and simple to explain: (1) start from a representation learned without labels, (2) fine-tune for classification, (3) keep a self-supervised auxiliary task available at test time, (4) adapt on unlabeled test data, and (5) compare performance with and without adaptation.

This document adapts that pipeline to the new constraints: **CIFAR-10**, a **ResNet-18 CNN**, and **rotation** as the distribution shift.

7 Dataset and distribution shift

Dataset: CIFAR-10 (32×32 RGB, 10 classes).

Training data: Standard CIFAR-10 train split with standard augmentations (random crop, horizontal flip, normalization).

Test shift: Rotation as a controlled distribution shift. At test time, each image is rotated by $0°$, $90°$, $180°$, or $270°$. This creates a systematic change in geometry while preserving the class label.

8 Model and losses

Backbone: A ResNet-18 CNN for a strong, stable baseline with good speed/accuracy trade-off.

Heads:

- Classification head for CIFAR-10 (10-way).
- Rotation head for the self-supervised task (4-way: 0, 90, 180, 270).

Classification loss: $\mathcal{L}_{\text{cls}} = \text{CrossEntropy}(y, \hat{y})$.

Rotation loss: $\mathcal{L}_{\text{rot}} = \text{CrossEntropy}(r, \hat{r})$.

Training objective:

$$\mathcal{L}_{\text{train}} = \mathcal{L}_{\text{cls}} + \lambda_{\text{rot}} \, \mathcal{L}_{\text{rot}}. \tag{1}$$

9 Training and test-time adaptation

9.1 Source training (no adaptation)

Train the model on CIFAR-10 with the combined loss. This produces a classifier head and a rotation head attached to the same backbone.

9.2 Classical TTT at test time

For each test batch (rotated data):

1. Create rotated versions of the batch and their rotation labels.
2. Run a small number of gradient steps on $\mathcal{L}_{\text{rot}}$ only.
3. Freeze the classifier head and adapt only the backbone (or last blocks) + rotation head.
4. After adaptation, run classification on the original rotated batch and record accuracy.

Note on backpropagation at test time: TTT does use backpropagation, but only on the self-supervised rotation loss. No class labels are used, and the classifier head stays fixed, so this is unsupervised adaptation rather than label leakage.

10 Baselines and evaluation

We must compare at least two conditions:

- **Baseline (no TTT):** Evaluate the trained classifier on rotated test data.
- **TTT:** Adapt on the rotation loss at test time, then classify.

Optional but recommended checks:

- Clean test accuracy (no rotation) to verify the classifier is healthy.
- Multiple rotation severities (fixed 90 vs random 0/90/180/270) to show robustness trends.

Primary metric: Top-1 accuracy.

11 Mapping to the current project structure

The repo already has a clean skeleton. The minimal additions are:

- `src/datasets.py`: add CIFAR-10 specs and rotation-wrapped dataset.
- `src/models/`: add a ResNet-18 wrapper (or use torchvision directly).
- `src/train.py`: supervised training loop with rotation auxiliary loss.
- `src/test_time_training.py`: implement the TTT adaptation step.
- `configs/`: add CIFAR-10 + ResNet-18 configs for baseline and TTT.

12 Concrete task split for two people

Person A: Data + Model + Training

- Implement CIFAR-10 loaders and rotation augmentation wrappers.
- Build the ResNet-18 model and heads.
- Implement supervised training with $\mathcal{L}_{\text{cls}} + \lambda_{\text{rot}}\mathcal{L}_{\text{rot}}$.
- Save checkpoints and log clean test accuracy.

Person B: TTT + Evaluation + Reporting

- Implement TTT adaptation in `src/test_time_training.py`.
- Run evaluation on rotated test data with and without TTT.
- Produce comparison tables/plots and summarize gains and failure cases.
- Update `docs/project_tracking.md` with results and decisions.

13 Milestones (short-term)

1. Week 1: CIFAR-10 pipeline runs end-to-end, baseline accuracy on clean test set.
2. Week 2: Rotation shift evaluation and first TTT results.
3. Week 3: Ablations (TTT steps, learning rate, which layers adapted).
4. Week 4: Final report section with comparison tables and figures.

14 Key decisions to keep consistent

- Use ResNet-18 as the backbone for both SSL and TTT experiments.
- Keep the classifier head fixed during TTT for interpretability.
- Use only rotation labels at test time (no access to class labels).
- Report both clean and rotated accuracy to show the trade-off.